Arquitectura de Xarxes

2025-2026 3rd quarter

Lab 3 - Virtualization with containers (Docker)

In this LAB, we are going to learn the basic concepts of containerization (virtualization via containers) using Docker. Virtualization is the core technology behind cloud computing, used by AWS, Microsoft Azure, Google Cloud, and most of the global IT infrastructure today.

Note: we will use the same VM image we used for the other sessions (running a single VM is enough), which has Docker already installed. The VirtualBox network adapter for the VM should be NAT, as in the previous lab.

Submission instructions:

- You must deliver a report **(in pdf format)** that details the procedures you followed and the results you obtained in this lab.
- In the cover page of your document, include the names and NIAs from all the members of the group.
- The pdf must be named "AX_lab1_NIA1_NIA2_NIA3.pdf".
- Deadline: the next lab session (before it starts).

Lab work

1. Execute docker commands to verify installation:

```
# See arguments supported by Docker
sudo docker --help

# Check the version installed
sudo docker --version
sudo docker -v
```

2. Using Docker with sudo is not recommended for security and usability reasons: i) Running Docker with sudo gives it root-level privileges, meaning any Docker container or command has full access to your system. ii) If a malicious container escapes, it could compromise your entire system.

   By default, Docker requires sudo. Fortunately, there is a safer alternative: adding our user to the docker group (`$USER` will be automatically substituted by your user name):

```
# Add our current user to the docker group
sudo usermod -aG docker $USER

# Now reboot the VM to apply the changes
sudo shutdown now

# - - - AFTER REBOOT - - -

# Verify Docker works without sudo
docker -v
docker run hello-world
```

3. Temporarily uninstall `flask` and `jsonify` python3 packages if already present, to illustrate how containers work on top of the hosting system (you can install them again after completing the lab). **Remember this step, as it will be important later.**

```
sudo pip3 uninstall flask jsonify
```

4. Create an arbitrary folder where you will put your configuration files. Copy our old, familiar flask python web server—available in the Moodle (Flask.py)—under the newly created folder. In the same folder, create the dockerfile as explained in the theory session, and make sure you use the correct *.py* file name.

   Note 1: apart from the already used `gedit` command, an alternative for managing files is the already installed `nano` utility. Feel free to use whichever you prefer.

```
# Edit, then press Ctrl+O to save, Ctrl+X to exit
nano <FILENAME>
```

   Note 2: another useful command is "cat". It basically outputs the content of a file, which is useful for quick checks:

```
cat Flask.py
```

5. Compile (build) the image using "restapi" for the image name:

```
docker build -t restapi .
```

   Note: the `-t` flag is for setting the image name. Remember that, in case you don't know/remember what a flag is used for, you can always use the `--help` flag:

```
docker build --help
```

   Check that the image has been successfully created with:

```
docker images
```

   With this output, you can see the size of each of the available images in the host. Explain what factors you think affect the size of a Docker image. Is it better to have a bigger image or a smaller one?

6. Create a network for your containers (again, you can use any other net name):

```
docker network create --subnet=172.18.0.0/16 dockerNet
```

   Check that the network has been successfully created.

```
docker network ls
```

   You will see that Docker already presents additional networks apart from the one we just created. No need to modify/delete them.

   Also, it is worth to check the network interfaces of our host:

```
ifconfig
```

   Note: another alternative to `ifconfig` that is widely used in Linux systems, is the command `ip`:

```
ip a
```

From the ifconfig/ip command output, you can notice that a new interface with the IP "172.18.0.1" has been created. This interface connects our host with the potential Docker containers that will be created within this network.

I guess you are curious about the docker0 interface. No need to worry about it. This is a default interface created by Docker at installation time. Such an interface is related with the network "bridge" that you saw from the `docker network ls` command output. We won't be using that network in our lab, as we want to use a different and dedicated one, although we could.

7. Start a container for your image. Parameter `--detach` (or `-d`) executes it in background mode (will not stop at execution), `--rm` removes container after finishing, `--publish` (or `-p`) specifies the ports that will be open/used, `--net` uses the network named in previous point, `--ip` associates the address to the image, and `--name` is the alias associated to the container, it can be other than restapiTest

```
docker run --detach --rm --publish 5200:5200 --name restapiTest --net
dockerNet --ip 172.18.0.2 restapi
```

Note: you can try to create another container using the same IP. Let's see what happens. You can also try using a different subnet (e.g: 172.20.0.2). You will see that Docker doesn't allow you to do it in either cases. Docker checks that the IP you provided is within the specified subnet, and that is not already in use.

8. The running container runs the Flask web server application, but instead of the app being run in the host directly, it's running on a dedicated container.

   List the running containers:

```
docker container ls
```

   We could inject some commands on the container:

```
docker exec restapiTest ls
docker exec restapiTest mkdir test
docker exec restapiTest ls
```

   In theory, the container should have assigned the IP that we have specified (172.20.0.2) before. Let's check it.

   We can access the container making use of the command "sh" and the flags -it. What do these flags mean?

```
docker exec -it restapiTest sh
```

   Once inside, let's check the network configuration:

```
docker exec -it restapiTest sh
> apt update && apt install net-tools
> ifconfig
```

   You can now exit the container with "exit". Run again the ifconfig command in the host. Can you explain why the output is different? Can the host and the container be

seen as different entities?

9. Now, let's check if the Flask web server is working fine. Let's verify this with the usual wget test (now with and `--progress=dot` not `--q`):

```
docker container ls

wget -O - --progress=dot http://172.18.0.2:5200/system
```

   Another alternative to the previous command is `curl`. They are equivalent, so again, it is up to you to use one or another.

```
curl http://172.18.0.2:5200/system
```

   Notice that the same web server **cannot** be run directly on the native host:

```
sudo python3 Flask.py
```

   This happens because the python packages flask and jsonify are not available in the host (as we uninstalled them at the beginning of the lab). Despite this, the Docker container is able to run the app (meaning that those packages are available on it).

   Is this the expected behaviour? Can you give an explanation to this?

10. Fetch the logs of our newly created container, and add a screenshot of them. Remember, that if you don't know a command, you can use `docker --help`, and try to derive it from that output. Another option is simply using Google.

11. Now, let's fetch the container logs in real-time. Add an additional flag to the command used in the previous question. Then, execute again the wget/curl test in a different terminal. Add a screenshot of what you observe.

12. Stop your container using the name you assigned (you could also use the "CONTAINER ID"):

```
docker container stop restapiTest
```

13. So far, we have seen that we can easily run applications making use of containers. We have deployed a Flask webserver, and we extracted some logs from it. Depending on the app, log storing could be mandatory. Think of a bank app, where all the connections (which IP, from where, at which time) should be stored.

    Stop the container (this should imply that the container is deleted as well). Check that the container is deleted. Now, run it again. Next, check the logs. Can you explain what is happening?

14. Now, let's try to give a solution to the previous issue. We will update the Flask application image so that logs are automatically stored in a specific location (in the folder /app/logs within the container).

    First, update your Dockerfile to create the directory "/app/logs". To do this, add this line in the RUN part:

```
RUN mkdir /app/logs
```

Next, update the CMD line so that all output of Flask server is saved in file flask.log in that directory (notice we use a different syntax of the CMD section):

```
CMD python3 Flask.py > /app/logs/flask.log 2>&1
```

Build your image again as we did in step 5 using the name "restapi-modified". Check that the image has been successfully created with `docker images`.

Now, create a docker container from this image, but now add the flag `-v /tmp:/app/logs` to the `docker run` command. You can check the meaning of `-v` taking a look at https://docs.docker.com/storage/bind-mounts/. Explain what is the purpose of `-v /tmp:/app/logs`.

Note: don't forget to use the new image name. Otherwise, you will still be creating instances of the "old restapi" image!

Now, make one or more wget/curl requests to generate some logs. Next, verify in /tmp directory of the host the existence of a new file "`flask.log`". What does it contain and why is it there? Will the logs be lost in case the container gets crashed/restarted?

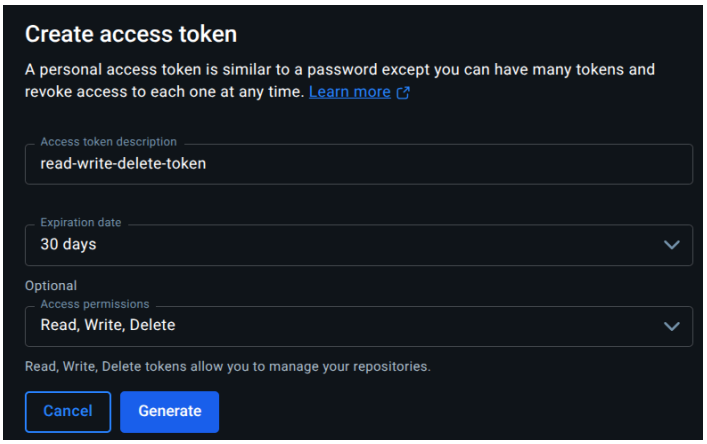
15. Cool, we're learning fast how containers work. Now, let's say that you want to share your newly developed app with other people. A great approach is uploading the Docker image we just created; restapi-modified, to a Docker registry.

    A Docker registry is basically the place where container images are stored and downloaded. There are many solutions available on the internet, but one, easy to use, is called DockerHub.

    Thus, create an account in https://hub.docker.com. We will use it to upload the docker image to the repository and make it available to anyone – also to the teacher of the session.

    When your account is ready, let's create a token that will be used to upload the image later on. To do so, go to Accounts settings/Personal access tokens as shown below.

    It is important that your access permissions include at least "Write", as otherwise, we won't be able to upload/push images.



Note: using tokens is a common security practice, as instead of using our current password to manage images, is it safer to just use a token, so in case the token leaks, we don't lose our account.

Once the token is set, follow the instructions that appear, to log into DockerHub from your host:

```
docker login -u <your-username>
<Paste-the-generated-token>
```

In order to upload/push the image into Dockerhub, as a pre-step we should re-tag (rename) the image with your account, repository and a tag, and finally push it to docker hub.

```
docker tag restapi-modified <your user>/restapi-modified:latest
docker push <your user>/restapi-modified:latest
```

Note: it is not necessary to create the repository beforehand, the push operation will do it for you.

The image can be tagged as explained here, or as in the theory slides (simply building the image with the exact name); an example would be `adrianpino/restapi-modified:latest`.

After pushing your image, you will see that it is available in the section "My Hub" (https://hub.docker.com/repositories/<your-username>). Add a screenshot of it. Note that the image is available if you search it. As the image is public (and not private), other groups (and basically, everyone) can see it and download it.

Congratulations, you've successfully published your first container app in DockerHub! Your teacher will download it, and check if it works.

16. As a last (optional) step, you can clean up the whole docker environment (and free up disk space!), remember that after that your network, containers and images will no longer be available:

```
docker system prune
```